Rapport de Projet

Assembleur x86 32 bits – MASM32

Explorateur de répertoire récursif (CLI et GUI)

O. Denis

[Université]

Février 2026

1 Introduction

Ce projet consiste en un explorateur de répertoire récursif développé en assembleur x86 32 bits sous MASM32. Deux versions ont été réalisées : une interface en ligne de commande (CLI) et une interface graphique Windows (GUI). Les deux implémentations parcourent une arborescence de répertoires de façon itérative grâce à une pile statique, sans recourir à la récursivité native du langage (absence d'appels récursifs de fonctions).

2 Partie CLI

2.1 Architecture

2.1.1 Segment `.data` (données initialisées)

Le segment `.data` contient les chaînes de caractères nécessaires à l'affichage et aux opérations de recherche :

- `adr1B` : invite affichée à l'utilisateur (`"Path:  "`)
- `msgDirectory` : en-tête affiché avant chaque répertoire visité
- `searchPattern` : motif `\*.*` utilisé comme argument de `FindFirstFileA`

2.1.2 Segment `.data?` (données non initialisées)

- `path` : chemin saisi par l'utilisateur (260 octets : 256 + lecteur + deux-points + slash + null)
- `searchPath` : chemin concaténé avec `\*.*` pour `FindFirstFileA`
- `findData` : structure `WIN32_FIND_DATAA` (320 octets)
- `stackPaths` : pile statique (60 × 260 octets = 15 600 octets)
- `stackPtr` : indice du sommet de pile
- `currentPath`, `tempBuffer` : buffers de travail pour la concaténation

Le choix d'une pile statique évite les débordements de pile d'appels. La profondeur maximale de 60 niveaux couvre la très grande majorité des arborescences Windows réelles.

2.2 Fonctionnement

2.2.1 Récupération des handles d'entrée/sortie

```
push -11            ; STD_OUTPUT_HANDLE
call GetStdHandle   ; kernel32.dll -- retourne le handle dans eax
mov stdOut, eax     ; sauvegarder le handle stdout

push -10            ; STD_INPUT_HANDLE
call GetStdHandle
mov stdIn, eax      ; sauvegarder le handle stdin
```

`GetStdHandle` est appelée avec la constante `-11` (stdout) puis `-10` (stdin). Le résultat est stocké dans `eax` puis sauvegardé en mémoire. Toutes les opérations d'affichage (`WriteFile`) et de lecture (`ReadFile`) nécessitent un handle valide.

2.2.2 Lecture et nettoyage du chemin

Après lecture du chemin via `ReadFile`, les 2 derniers octets (CR LF) sont supprimés :

```
mov ecx, bytesRead        ; nombre d'octets lus
sub ecx, 2                ; supprimer CR LF
mov byte ptr [path + ecx], 0  ; null-terminer la cha ne
mov pathLength, ecx       ; sauvegarder la longueur
```

2.3 Boucle principale

La boucle `boucle_stack` dépile un chemin, liste le contenu du répertoire correspondant, empile les sous-répertoires trouvés, puis recommence jusqu'à ce que la pile soit vide.

```
boucle_stack:
    mov eax, stackPtr
    cmp eax, 0                ; pile vide ?
    jz fin_recursion          ; oui -- tous les repertoires sont
       traites

    sub stackPtr, 260         ; depiler : reculer d'un slot
    mov eax, stackPtr         ; eax = offset du chemin depile
    lea esi, [stackPaths + eax]
    mov edi, offset currentPath
    mov ecx, 260              ; copier au plus 260 octets
    ; ... (boucle de copie octet par octet)
    jmp boucle_stack
```

2.4 Détection des sous-répertoires

L'attribut `FILE_ATTRIBUTE_DIRECTORY` (bit 4 = `0x10`) est testé dans le premier DWORD de la structure `WIN32_FIND_DATAA` :

```
mov eax, dword ptr findData ; dwFileAttributes (offset 0)
and eax, 10h                ; masquer le bit repertoire
jz pas_dossier              ; flag absent => fichier normal
```

Note : la lecture via `dword ptr findData` (adresse symbolique) plutôt qu'une adresse directe résolue à la compilation a été nécessaire pour éviter la corruption de l'objet `.obj` lors des premières phases de test.

2.5 Filtrage de `.` et `..`

Les entrées `.` (répertoire courant) et `..` (répertoire parent) sont systématiquement présentes dans tout répertoire Windows. Sans filtrage, la boucle devient infinie :

```
lea esi, [findData+44]    ; cFileName (offset 44 dans WIN32_FIND_DATAA)
mov al, byte ptr [esi]
cmp al, '.'
jne pas_point             ; premier caractere != '.' -- traiter normalement

mov al, byte ptr [esi+1]
cmp al, 0                 ; juste "." seul ?
je fichier_suivant        ; sauter

cmp al, '.'
jne pas_point             ; deuxieme caractere != '.' -- traiter normalement

mov al, byte ptr [esi+2]
cmp al, 0                 ; juste ".." seul ?
je fichier_suivant        ; sauter
```

3 Partie GUI

3.1 Architecture

3.1.1 Segment `.data` (données initialisées)

- `szClassName` : nom de la classe de fenêtre (`"FileExplorerApp"`)
- `szAppName` : titre de la fenêtre principale (`"Recursive File Explorer"`)
- `szEditClass`, `szButtonClass`, `szStaticClass` : noms des classes de contrôles Windows
- `szOKButton` : texte du bouton (`"Browse"`)
- `szDefaultInput` : chemin par défaut dans la TextBox (`"C:\Users\user"`)
- `searchPattern` : motif `\*.*` pour la recherche

3.1.2 Segment `.data?` (données non initialisées)

- `hInstance`, `hWnd`, `hEdit`, `hButton`, `hResult` : handles de la fenêtre et des contrôles
- `path` : chemin saisi (260 octets)
- `searchPath` : chemin + `\*.*` (520 octets)
- `findData` : structure `WIN32_FIND_DATAA` (320 octets)
- `stackPaths` : pile des chemins ($60 \times 260 = 15\,600$ octets)
- `stackDepths` : pile des profondeurs ($60 \times 4 = 240$ octets)
- `resultBuffer` : buffer de 32 000 octets pour accumuler les résultats

- `resultPtr` : position courante d'écriture dans `resultBuffer`

3.2 Fonctionnement général

Contrairement à la version CLI qui lit et écrit directement sur la console, la version GUI attend que l'utilisateur saisisse un chemin dans la zone de texte dédiée et clique sur le bouton **Browse**. La recherche récursive fonctionne ensuite de manière identique à la version CLI, à l'exception de l'affichage : tous les résultats sont accumulés dans `resultBuffer` et envoyés en une seule opération vers la zone de résultats via `SetWindowTextA`.

3.3 Initialisation

Au démarrage, `GetModuleHandleA` récupère le handle de l'instance courante. La classe de fenêtre est enregistrée via `RegisterClassExA` avec la structure `WNDCLASSEX` remplie manuellement (style `CS_HREDRAW|CS_VREDRAW`, curseur flèche, fond `COLOR_BTNFACE`). La fenêtre principale est ensuite créée (900 × 600 pixels) par `CreateWindowExA`, puis rendue visible par `ShowWindow`.

3.4 Création des contrôles – `CreateControls`

Lors de la réception du message `WM_CREATE`, `WndProc` appelle `CreateControls` qui instancie les trois contrôles de l'interface :

- **TextBox d'entrée (ID 1000)** : zone de saisie du chemin ; le chemin par défaut est injecté immédiatement via `SetWindowTextA`.
- **Bouton « Browse » (ID 1001)** : déclenche la récursion.
- **Zone de résultats (ID 1002)** : TextBox multiligne en lecture seule avec barres de défilement.

Tous les contrôles sont créés avec `WS_CHILD | WS_VISIBLE` pour être rattachés à la fenêtre parente et visibles immédiatement.

3.5 Déclenchement de la recherche – `ScanAndDisplay`

Lors d'un clic sur le bouton, `WndProc` reçoit `WM_COMMAND`. Si l'identifiant vaut `1001`, `ScanAndDisplay` est appelée. Cette fonction :

1. Lit le contenu de la TextBox avec `GetWindowTextA` (260 caractères max).
2. Réinitialise les variables de contrôle (`resultPtr`, `fileCounter`, `currentDepth`) et les deux piles.
3. Ajoute l'en-tête du chemin racine via `AddRootHeader`.
4. Empile le chemin initial avec la profondeur 0.
5. Lance la boucle principale, identique à la version CLI.

3.6 Affichage des résultats – `AddFileNameToResult` / `AddRootHeader`

`AddFileNameToResult` écrit dans `resultBuffer` en utilisant `resultPtr` comme curseur. Elle insère d'abord autant de tabulations (ASCII 9) que la valeur de `currentDepth`, produisant l'indentation hiérarchique. Le nom du fichier ou du dossier (`cFileName`, offset 44 dans `WIN32_FIND_DATAA`) est copié octet par octet, suivi d'un saut de ligne CR LF.

Une fois la récursion terminée, `SetWindowTextA` envoie l'intégralité de `resultBuffer` à la zone de résultats. L'affichage se fait donc en une seule opération, contrairement à la version CLI qui écrit ligne par ligne avec `WriteFile`.

4 Comparaison CLI / GUI

Aspect	CLI	GUI
Entrée chemin	`ReadFile` (stdin)	`GetWindowTextA` (TextBox)
Affichage résultats	`WriteFile` ligne par ligne	`SetWindowTextA` en une passe
Suivi de profondeur	Non (pas d'indentation)	Oui (pile `stackDepths`)
Buffer résultats	Non (sortie directe console)	`resultBuffer` 32 000 octets
Subsystem linker	`CONSOLE`	`WINDOWS`

5 Difficultés rencontrées

1. **Corruption du fichier `.obj`** : lors des premières tentatives d'accès à `dwFileAttributes`, l'utilisation d'une adresse directe résolue à la compilation faisait disparaître le fichier objet généré. La solution a consisté à utiliser `dword ptr findData` (adresse symbolique).
2. **Boucle infinie sur `.` et `..`** : sans filtrage explicite de ces deux entrées, la pile débordait immédiatement. L'ajout du double test (premier et deuxième caractère) a résolu le problème.
3. **Gestion du séparateur ** : lors de la construction du chemin de recherche, il a fallu vérifier si le chemin fourni par l'utilisateur se terminait déjà par \ avant d'en ajouter un, afin d'éviter les chemins malformés (`C:\\*.*`).
4. **Limite du buffer de résultats (GUI)** : `resultBuffer` est dimensionné à 32 000 octets. Des gardes (`cmp resultPtr, 31800`) sont en place pour éviter tout débordement.

6 Conclusion

Ce projet a permis de mettre en pratique la programmation assembleur x86 32 bits sous Windows en interagissant directement avec l'API Win32. Les deux versions (CLI et GUI) illustrent comment un même algorithme de parcours récursif peut être adapté à deux paradigmes d'interface différents, tout en réutilisant la même structure de pile itérative. La version GUI apporte en outre la gestion de la profondeur d'arborescence pour l'indentation visuelle des résultats.